QCon
全球软件开发大会

InfoQ 技术大会

AReaL 2.0

让 Agent 越用越强：构建 Agent 在线强化学习闭环

袁彬航

香港科技大学计算机科学与工程系助理教授
AReaL 开源社区负责人

2026年10月20日



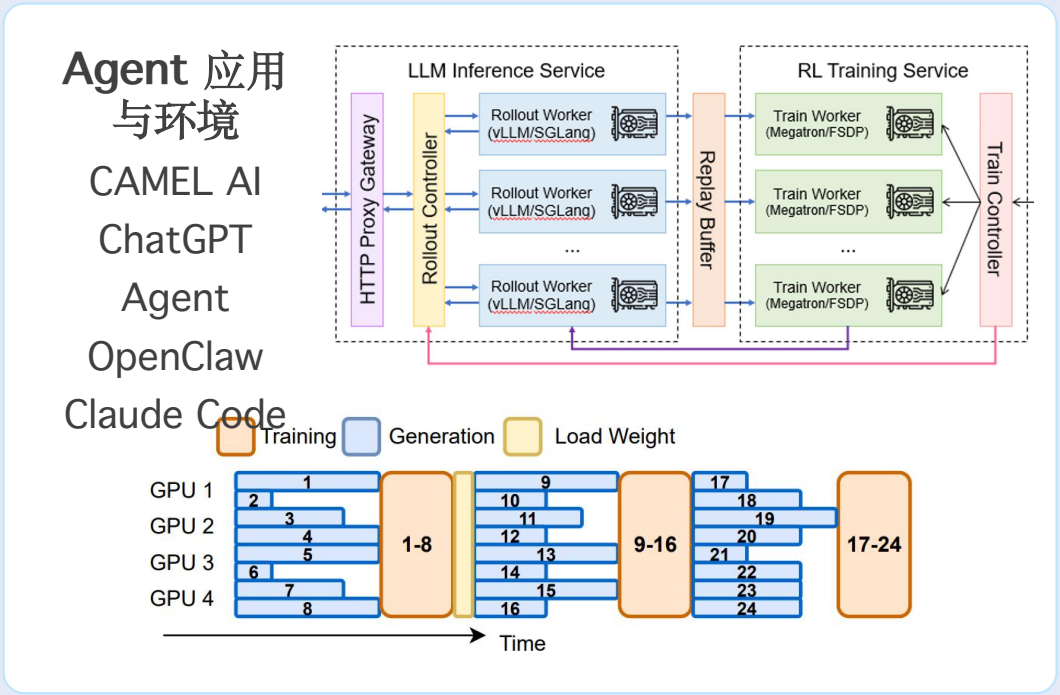
AReaL

01

AReaL 开源社区

面向大规模 Agent 学习的开放基础设施

AReaL：开源社区



PyTorch 生态项目

2026年5月获接纳
正在申请基金会托管

什么是 AReaL？

可扩展的 Agent 强化学习（RL）基础设施，连接基础模型训练与现代 Agent 应用。
2025年2月，由蚂蚁集团与清华大学联合开源

- 灵活性 — 模块化架构，支持即插即用的 Agent 强化学习服务。
- 可扩展性 — 支持稳定的大规模全异步强化学习训练。
- 性能领先 — 在数学、代码、搜索及 Agent 任务中取得领先表现。
- 研究友好 — 支持四类主要强化学习范式及 10 余种主流算法。
- PyTorch 生态项目 — 2026年5月获接纳，现正申请基金会托管。

AReaL：社区与治理

5.7K+	500+	100+	1,900+	4 → 14
GitHub 星标	代码分叉	代码贡献者	社区用户	发布以来的维护者人数

厂商中立的治理机制

- 维护者主导，治理规则见 GOVERNANCE.md。
- 14 位维护者来自清华、蚂蚁、华为、字节跳动、小米和腾讯六家机构。
- 每个 PR 须经两位维护者批准，其中一位为相关路径的代码负责人。
- 采用共识决策，无法达成共识时由首席维护者裁决。
- OpenSSF 最佳实践金徽章（项目编号 12770）。
- 已为向 Linux 基金会移交项目资产做好准备。

贡献者与维护者所属机构

维护者

清华大学　蚂蚁集团
华为　字节跳动
腾讯　小米

贡献者来自

微软　Boson AI
北京大学　香港中文大学
香港科技大学　UIUC

AReaL：版本演进与发布节奏

每次发布包含三类成果：

1. **版本化代码**
   附版本标签，可安装、可复现。
2. **可复现训练示例**
   包含配置与脚本的端到端训练方案。
3. **研究成果与论文**
   随每个里程碑发布实验结果。

v0.1 至 v2.0 历时 18 个月
经历推理强化学习、Agent 强化学习
与在线强化学习三代架构

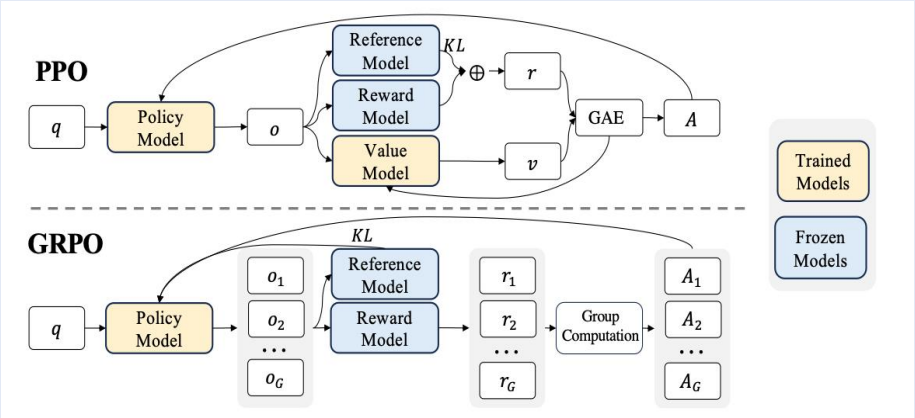
2026年8月	v2.0	在线强化学习模块化架构	在线强化学习
2026年6月		正式宣布加入 PyTorch 生态项目	
2026年3月	v1.0	Agent 架构正式发布	Agent 强化学习
2025年12月	v0.5	Agent 架构预览版	
2025年8月	v0.4	AReaL-lite 架构	推理强化学习
2025年6月	v0.3	全异步强化学习，吞吐量达 2.7×	
2025年3月	v0.2	领先的 7B 数学模型	
2025年2月	v0.1	首次发布，支持 GRPO	

02

AReaL 0.1
强化学习基础设施

面向大模型后训练的高效异步强化学习

大模型的强化学习后训练



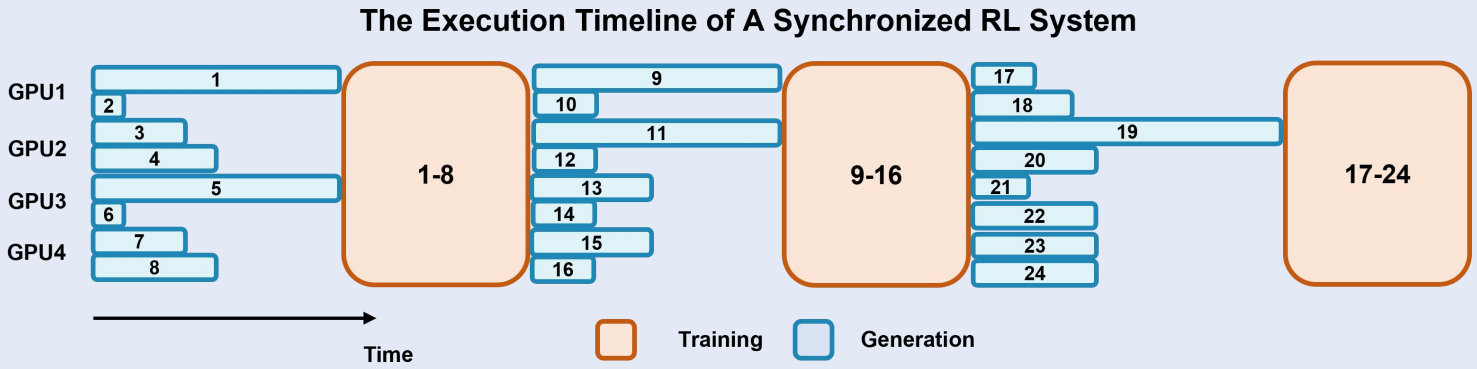
轨迹采样：策略大模型为一批提示词生成响应。

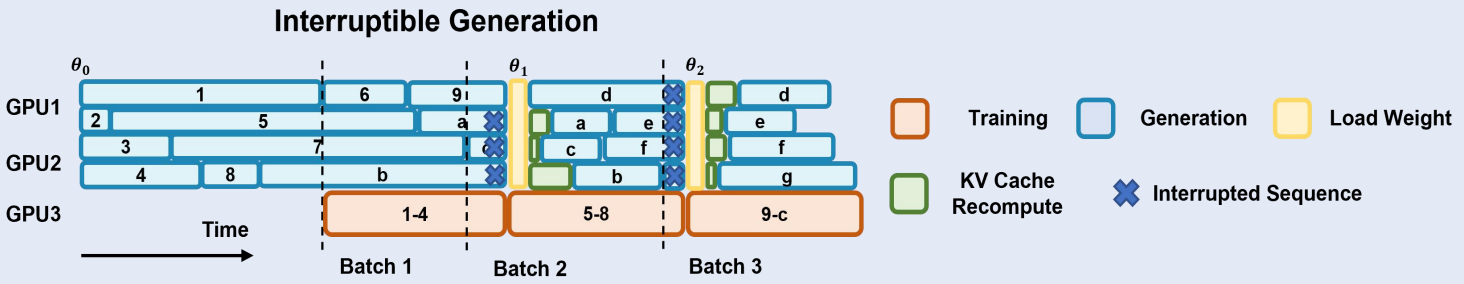
奖励计算：价值模型估计价值，参考策略提供对数概率，奖励模型或沙箱中的函数对响应评分。

模型训练：基于采集的数据与奖励，通过梯度优化更新策略。

AReaL：异步强化学习

异步强化学习提升大模型后训练效率







AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning

Wei Fu[12], Jiaxuan Gao[1], Xujie Shen[2], Chen Zhu[2], Zhiyu Mei[12], Chuyi He[2], Shusheng Xu[12], Guo Wei[2], Jun Mei[2], Jiashu Wang[3], Tongkai Yang[2], Binhang Yuan[3], Yi Wu[1]

[1] IIIS, Tsinghua University, [2] Ant Research, [3] HKUST
fuwth17@gmail.com, jxwuyi@gmail.com

Abstract

Reinforcement learning (RL) has become a trending paradigm for training large language models (LLMs), particularly for reasoning tasks. Effective RL for LLMs requires massive parallelization and poses an urgent need for efficient training systems. Most existing large-scale RL systems for LLMs are synchronous by alternating generation and training in a batch setting, where the rollouts in each training batch are generated by the same (or latest) model. This stabilizes RL training but suffers from severe system-level inefficiency. Generation must wait until the longest output in the batch is completed before model update, resulting in GPU underutilization. We present AReaL, a *fully asynchronous* RL system that completely decouples generation from training. Rollout workers in AReaL continuously generate new outputs without waiting, while training workers update the model whenever a batch of data is collected. AReaL also incorporates a collection of system-level optimizations, leading to substantially higher GPU utilization. To stabilize RL training, AReaL balances the workload of rollout and training workers to control data staleness, and adopts a staleness-enhanced PPO variant to better handle outdated training samples. Extensive experiments on math and code reasoning benchmarks show that AReaL achieves **up to 2.77× training speedup** compared to synchronous systems with the same number of GPUs and matched or even improved final performance. The code of AReaL is available at https://github.com/inclusionAI/AReaL/.

1 Introduction

Reinforcement learning (RL) has emerged as a new scaling paradigm for enhancing the capabilities of large language models (LLMs) by enabling thinking abilities [52]. Given a prompt, RL allows an LLM to generate thinking tokens before outputting a final answer, enabling *test-time scaling* [29, 47]. These thinking LLMs are named Large Reasoning Models (LRMs) and have been shown to have particularly strong capabilities on challenging reasoning problems, such as math [9, 5, 20], coding [3, 14, 15], logic puzzles [22, 34], and agentic tasks [23, 57].

Effective RL training often requires massive parallelization to derive a large batch of rollouts for sufficient exploration, which is the key to obtaining optimal model performance. For example, popular RL algorithms, such as PPO [42] and GRPO [43], often require an effective training batch of thousands of outputs [60, 61, 53]. Moreover, an LRM can generate tens of thousands of thinking tokens for each input prompt [6], further posing an urgent need for an efficient training system to run RL training on a large scale.

[NeurIPS 2025]

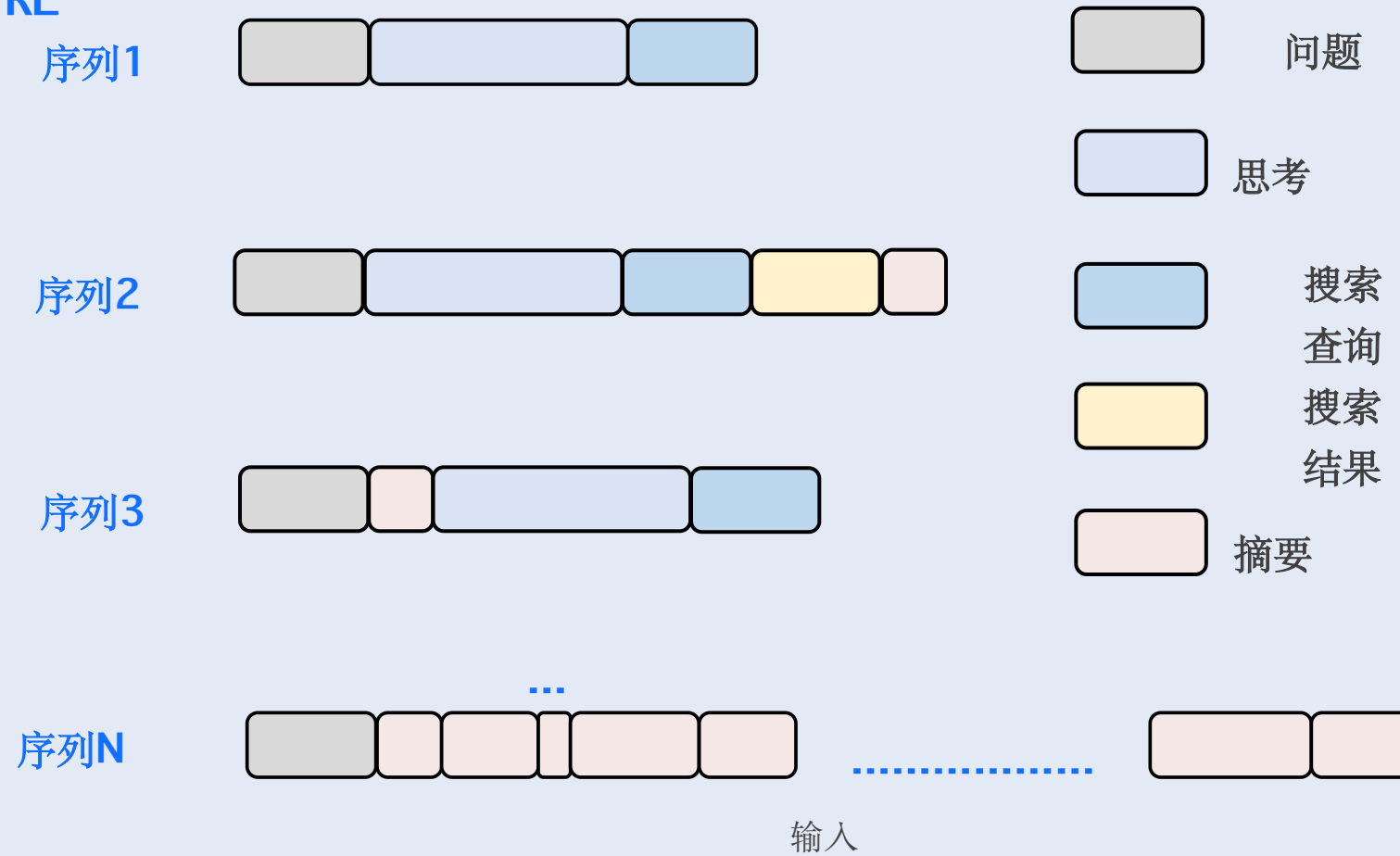
03

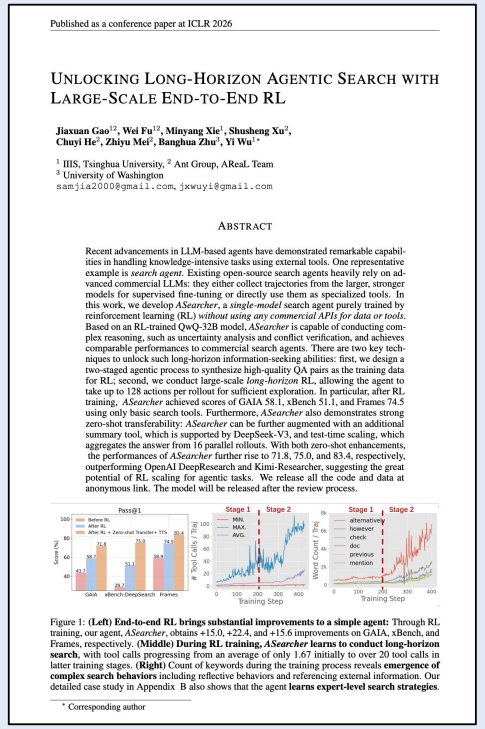
AReaL 1.0
Agent 强化学习

通过工具交互训练长程 Agent

ASearcher：长程 Agent 搜索

Beyond Ten Turns: Unlocking Long-Horizon Agentic Search with Large-Scale Asynchronous RL





Published as a conference paper at ICLR 2026

UNLOCKING LONG-HORIZON AGENTIC SEARCH WITH LARGE-SCALE END-TO-END RL

Jiaxuan Gao[1,2], Wei Fu[1,2], Minyang Xie[1], Shusheng Xu[2], Chuyi He[2], Zhiyu Mei[2], Banghua Zhu[3], Yi Wu[1,*]

[1] IIIS, Tsinghua University, [2] Ant Group, AReaL Team
[3] University of Washington
samjia2000@gmail.com, jxwuyi@gmail.com

ABSTRACT

Recent advancements in LLM-based agents have demonstrated remarkable capabilities in handling knowledge-intensive tasks using external tools. One representative example is *search agent*. Existing open-source search agents heavily rely on advanced commercial LLMs: they either collect trajectories from the larger, stronger models for supervised fine-tuning or directly use them as specialized tools. In this work, we develop *ASearcher*, a *single-model* search agent purely trained by reinforcement learning (RL) *without using any commercial APIs for data or tools*. Based on an RL-trained QwQ-32B model, *ASearcher* is capable of conducting complex reasoning, such as uncertainty analysis and conflict verification, and achieves comparable performances to commercial search agents. There are two key techniques to unlock such long-horizon information-seeking abilities: first, we design a two-staged agentic process to synthesize high-quality QA pairs as the training data for RL; second, we conduct large-scale *long-horizon* RL, allowing the agent to take up to 128 actions per rollout for sufficient exploration. In particular, after RL training, *ASearcher* achieved scores of GAIA 58.1, xBench 51.1, and Frames 74.5 using only basic search tools. Furthermore, *ASearcher* also demonstrates strong zero-shot transferability: *ASearcher* can be further augmented with an additional summary tool, which is supported by DeepSeek-V3, and test-time scaling, which aggregates the answer from 16 parallel rollouts. With both zero-shot enhancements, the performances of *ASearcher* further rise to 71.8, 75.0, and 83.4, respectively, outperforming OpenAI DeepResearch and Kimi-Researcher, suggesting the great potential of RL scaling for agentic tasks. We release all the code and data at anonymous link. The model will be released after the review process.

Figure 1: **(Left) End-to-end RL brings substantial improvements to a simple agent:** Through RL training, our agent, *ASearcher*, obtains +15.0, +22.4, and +15.6 improvements on GAIA, xBench, and Frames, respectively. **(Middle) During RL training, *ASearcher* learns to conduct long-horizon search**, with tool calls progressing from an average of only 1.67 initially to over 20 tool calls in latter training stages. **(Right)** Count of keywords during the training process reveals **emergence of complex search behaviors** including reflective behaviors and referencing external information. Our detailed case study in Appendix B also shows that the agent **learns expert-level search strategies**.

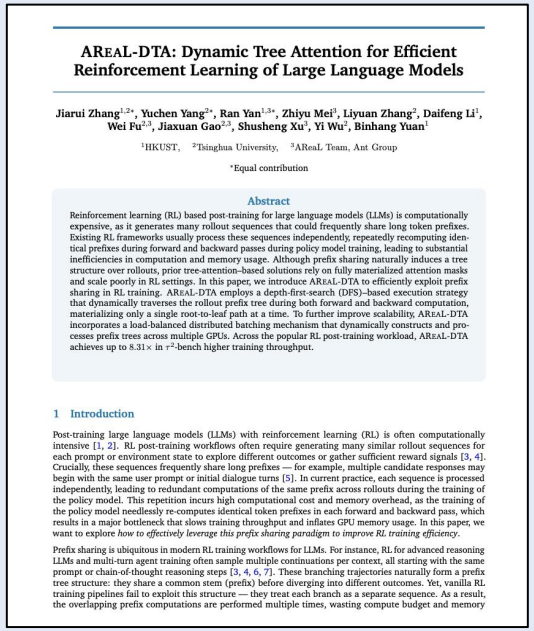
* Corresponding author

[ICLR 2026]

AReaL 系统优化：AReaL-DTA

AREAL-DTA: Dynamic Tree Attention for Efficient Reinforcement Learning of Large Language Models

- 复用采样轨迹的共享前缀，避免重复计算。
- 采用基于 DFS 的注意力机制遍历前缀树。
- 均衡 GPU 间的计算负载。



AREAL-DTA: Dynamic Tree Attention for Efficient Reinforcement Learning of Large Language Models

Jiarui Zhang[1,2*], Yuchen Yang[2*], Ran Yan[1,3*], Zhiyu Mei[3], Liyuan Zhang[2], Daifeng Li[1], Wei Fu[2,3], Jiaxuan Gao[2,3], Shusheng Xu[3], Yi Wu[2], Binhang Yuan[1]

[1]HKUST, [2]Tsinghua University, [3]AReaL Team, Ant Group

*Equal contribution

Abstract

Reinforcement learning (RL) based post-training for large language models (LLMs) is computationally expensive, as it generates many rollout sequences that could frequently share long token prefixes. Existing RL frameworks usually process these sequences independently, repeatedly recomputing identical prefixes during forward and backward passes during policy model training, leading to substantial inefficiencies in computation and memory usage. Although prefix sharing naturally induces a tree structure over rollouts, prior tree-attention–based solutions rely on fully materialized attention masks and scale poorly in RL settings. In this paper, we introduce AReaL-DTA to efficiently exploit prefix sharing in RL training. AReaL-DTA employs a depth-first-search (DFS)–based execution strategy that dynamically traverses the rollout prefix tree during both forward and backward computation, materializing only a single root-to-leaf path at a time. To further improve scalability, AReaL-DTA incorporates a load-balanced distributed batching mechanism that dynamically constructs and processes prefix trees across multiple GPUs. Across the popular RL post-training workload, AReaL-DTA achieves up to 8.31× in τ^2-bench higher training throughput.

1 Introduction

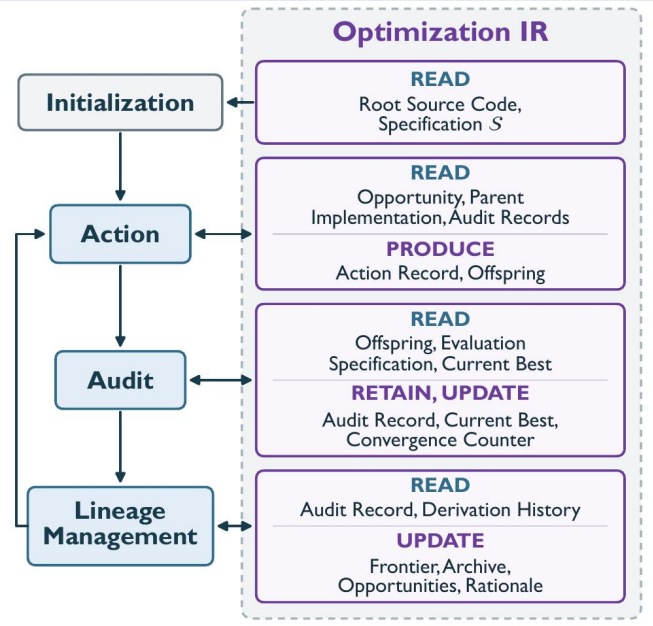
Post-training large language models (LLMs) with reinforcement learning (RL) is often computationally intensive [1, 2]. RL post-training workflows often require generating many similar rollout sequences for each prompt or environment state to explore different outcomes or gather sufficient reward signals [3, 4]. Crucially, these sequences frequently share long prefixes — for example, multiple candidate responses may begin with the same user prompt or initial dialogue turns [5]. In current practice, each sequence is processed independently, leading to redundant computations of the same prefix across rollouts during the training of the policy model. This repetition incurs high computational cost and memory overhead, as the training of the policy model needlessly re-computes identical token prefixes in each forward and backward pass, which results in a major bottleneck that slows training throughput and inflates GPU memory usage. In this paper, we want to explore *how to effectively leverage this prefix sharing paradigm to improve RL training efficiency*.

Prefix sharing is ubiquitous in modern RL training workflows for LLMs. For instance, RL for advanced reasoning LLMs and multi-turn agent training often sample multiple continuations per context, all starting with the same prompt or chain-of-thought reasoning steps [3, 4, 6, 7]. These branching trajectories naturally form a prefix tree structure: they share a common stem (prefix) before diverging into different outcomes. Yet, vanilla RL training pipelines fail to exploit this structure — they treat each branch as a separate sequence. As a result, the overlapping prefix computations are performed multiple times, wasting compute budget and memory

[ICML 2026]

AReaL 系统优化：AReaL-TIK

基于有状态 Agent 的统一强化学习内核优化

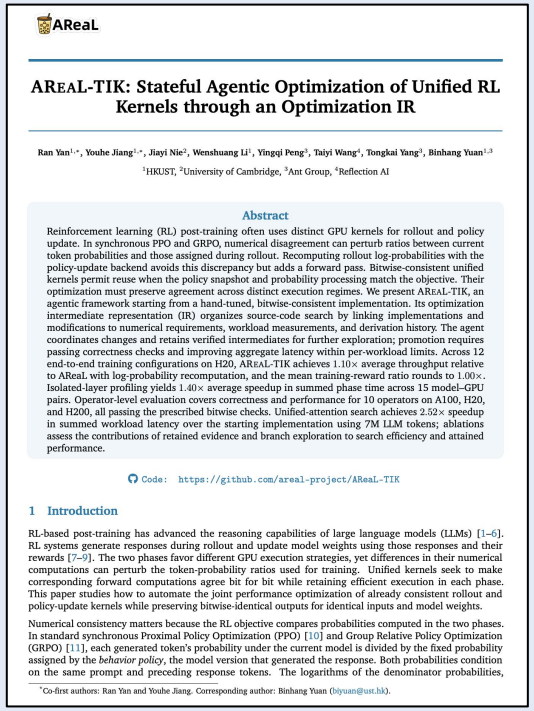


采样与更新的一致性

数值不一致会引入额外前向计算，以重算采样时的对数概率。

有状态内核优化

联合优化逐比特一致的内核。持久化中间表示（IR）保留代码、测量结果和已验证分支，供后续修改使用。



AReaL

AReaL-TIK: Stateful Agentic Optimization of Unified RL Kernels through an Optimization IR

Ran Yan[1,*], Youhe Jiang[1,*], Jiayi Nie[2], Wenshuang Li[1], Yingqi Peng[3], Taiyi Wang[4], Tongkai Yang[3], Binhang Yuan[1,3]

[1]HKUST, [2]University of Cambridge, [3]Ant Group, [4]Reflection AI

Abstract

Reinforcement learning (RL) post-training often uses distinct GPU kernels for rollout and policy update. In synchronous PPO and GRPO, numerical disagreement can perturb ratios between current token probabilities and those assigned during rollout. Recomputing rollout log-probabilities with the policy-update backend avoids this discrepancy but adds a forward pass. Bitwise-consistent unified kernels permit reuse when the policy snapshot and probability processing match the objective. Their optimization must preserve agreement across distinct execution regimes. We present AReaL-TIK, an agentic framework starting from a hand-tuned, bitwise-consistent implementation. Its optimization intermediate representation (IR) organizes source-code search by linking implementations and modifications to numerical requirements, workload measurements, and derivation history. The agent coordinates changes and retains verified intermediates for further exploration; promotion requires passing correctness checks and improving aggregate latency within per-workload limits. Across 12 end-to-end training configurations on H20, AReaL-TIK achieves 1.10× average throughput relative to AReaL with log-probability recomputation, and the mean training-reward ratio rounds to 1.00×. Isolated-layer profiling yields 1.40× average speedup in summed phase time across 15 model–GPU pairs. Operator-level evaluation covers correctness and performance for 10 operators on A100, H20, and H200, all passing the prescribed bitwise checks. Unified-attention search achieves 2.52× speedup in summed workload latency over the starting implementation using 7M LLM tokens; ablations assess the contributions of retained evidence and branch exploration to search efficiency and attained performance.

Code: https://github.com/areal-project/AReaL-TIK

1 Introduction

RL-based post-training has advanced the reasoning capabilities of large language models (LLMs) [1–6]. RL systems generate responses during rollout and update model weights using those responses and their rewards [7–9]. The two phases favor different GPU execution strategies, yet differences in their numerical computations can perturb the token-probability ratios used for training. Unified kernels seek to make corresponding forward computations agree bit for bit while retaining efficient execution in each phase. This paper studies how to automate the joint performance optimization of already consistent rollout and policy-update kernels while preserving bitwise-identical outputs for identical inputs and model weights.

Numerical consistency matters because the RL objective compares probabilities computed in the two phases. In standard synchronous Proximal Policy Optimization (PPO) [10] and Group Relative Policy Optimization (GRPO) [11], each generated token's probability under the current model is divided by the fixed probability assigned by the *behavior policy*, the model version that generated the response. Both probabilities condition on the same prompt and preceding response tokens. The logarithms of the denominator probabilities,

*Co-first authors: Ran Yan and Youhe Jiang. Corresponding author: Binhang Yuan (biyuan@ust.hk).

[arXiv 2609.35140]

平均训练吞吐量 1.10×

平均奖励比值 1.00×（四舍五入）

12 组 H20 配置；对照为重算对数概率的 AReaL

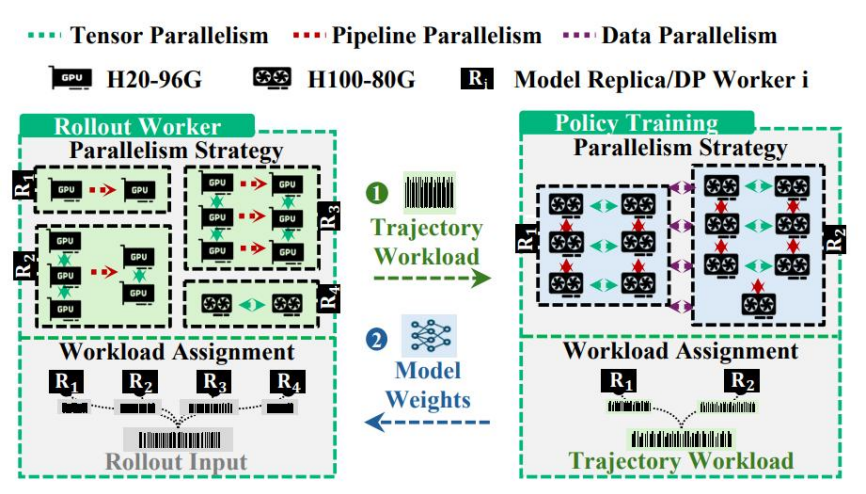
注意力计算加速比 2.52×

采用相同的初始实现
700 万个大模型 Token

AReaL 系统优化：AReaL-Hex

面向异构 GPU 的异步强化学习训练

将受带宽限制的采样与受算力限制的训练分配到合适的 GPU。
在策略滞后有界的约束下，选择并行执行方案并重新划分资源。



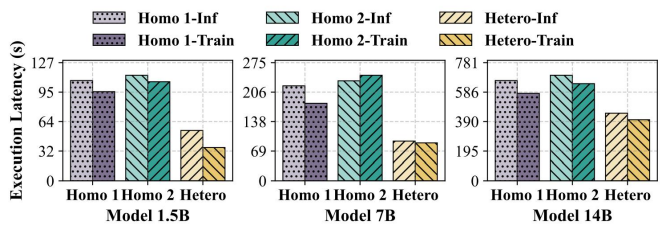
训练吞吐量最高达 1.50×

在相同总预算下

吞吐量相当，成本更低

Model	System	Throughput	Cost
1.5B model	AReaL (H800)	1.84×10^4 t/s	$126.72/h
	AReaL-Hex	1.79×10^4 t/s	$86.64/h
7B model	AReaL (H800)	5.98×10^3 t/s	$126.72/h
	AReaL-Hex	6.00×10^3 t/s	$86.64/h
14B model	AReaL (H800)	2.16×10^3 t/s	$126.72/h
	AReaL-Hex	2.30×10^3 t/s	$86.64/h

相同预算下的延迟



每小时成本：$86.64 vs $126.72

在吞吐量相当时



AReaL-Hex: Accommodating Asynchronous RL Training over Heterogeneous GPUs

Ran Yan[1*], Youhe Jiang[1*], Tianyuan Wu[1], Jiaxuan Gao[2], Zhiyu Mei[3], Wei Fu[2], Haohui Mai[3], Wei Wang[1], Yi Wu[2], Binhang Yuan[1]

[1]HKUST, [2]Tsinghua University, [3]Ant Group

[*]Equal contribution

arXiv:2511.00796v1 [cs.DC] 2 Nov 2025

Abstract

Maximizing the training throughput and cost-efficiency of reinforcement learning (RL) for large language models (LLMs) is essential to democratize this advanced technique. One promising but challenging approach is to deploy such a computational workflow over heterogeneous GPU clusters. Unlike conventional large-scale LLM pretraining, RL training generally decomposes into three coupled stages, i.e., rollout generation, reward computation, and policy/value updates, which exhibit markedly different compute intensities, memory footprints, and communication patterns. Recent research shows that fully asynchronous RL training can disaggregate these stages across disjoint hardware pools without sacrificing training stability, creating a great opportunity for real-world heterogeneous deployment. Towards this end, we present AReaL-Hex, a heterogeneity-aware asynchronous RL training system that effectively schedules how to execute rollout generation and policy model training over heterogeneous GPU clusters while enforcing data staleness bounds. Concretely, AReaL-Hex uses a two-phase scheduler: (i) a constrained search with mixed-integer linear programming to select per-stage parallelization strategies and workload assignments given a resource budget, and (ii) a graph-partitioning step that allocates heterogeneous GPUs and interconnects to maximize end-to-end throughput. Built atop a fully asynchronous RL architecture, AReaL-Hex maps HBM-I/O-bound generation and compute-bound optimization to more cost-efficient resources and balances their producer–consumer interactions to avoid both idleness and stale rollout trajectories. On the mathematical reasoning task with various model scales (1.5B, 7B, and 14B), compared to homogeneous deployments of state-of-the-art asynchronous RL systems: (i) When maintaining the same total budgets, AReaL-Hex delivers up to 1.50× higher training throughput; (ii) When achieving the same training throughput, AReaL-Hex results in up to 1.46× reduction in training cost.

1 Introduction

Reinforcement learning (RL) has emerged as a popular approach for enhancing large language models (LLMs) across various tasks [1], including math [2], code generation [3], agentic tool use [4], etc, which becomes essential to advance the development of AI. Unlike conventional large-scale pretraining by standard stochastic gradient-based optimization, RL training comprises multiple coupled stages, where each stage includes heterogeneous compute intensities, memory footprints, and communication patterns [5]; furthermore, recent study suggests that asynchronous RL paradigm can effectively disaggregate these different stages over different set of hardwares without compromising the stability in RL training [6, 7]. Such an intrinsic computational heterogeneity under the disaggregated infrastructure naturally provides a great opportunity to deploy such a

[arXiv 2511.00796]

04

AReaL 2.0 在线强化学习

将 Agent 交互接入持续在线学习

现有能力的缺口

静态 Agent，人工改进流程

数以百万计的 Agent 交互，仍需依靠人工工程迭代。

人工改进流程：



1. 检查执行轨迹
2. 定义评测基准
3. 调整提示词与工具
4. 强化学习 / 微调
5. 重新部署　　数周 → 数月



我们的洞察：

企业级 Agent 自我进化的瓶颈，并非模型规模、提示词技巧或缺少新的强化学习算法。

真正缺少的是支撑 Agent 在线强化学习的系统底座。

Agent 自我进化的三大支柱

三层协同，将 Agent 交互转化为受治理的学习过程。

1 ATDP

Agent 轨迹数据协议

采集什么数据

标准化、厂商中立，满足强化学习单步训练要求的轨迹格式。跨 Agent 框架记录类型明确、可审计、支持贡献归因的事件。

2 数据代理

全链路
Agent 数据代理

如何采集数据

在稳定边界截获模型调用、工具、检索、记忆及人工反馈；经脱敏、持久化和确定性重放，形成受治理的学习数据。

3 演化控制

Agent 演化
控制平面

何时学、如何学

根据轨迹统计选择改进对象：记忆、技能、执行框架（Harness）、工具接口或模型权重。每一步均设准入检查，支持安全回滚。

Agent 轨迹数据协议（ATDP）

> 标准化、厂商中立的轨迹格式，为强化学习完整保留决策生命周期。

数据结构原型

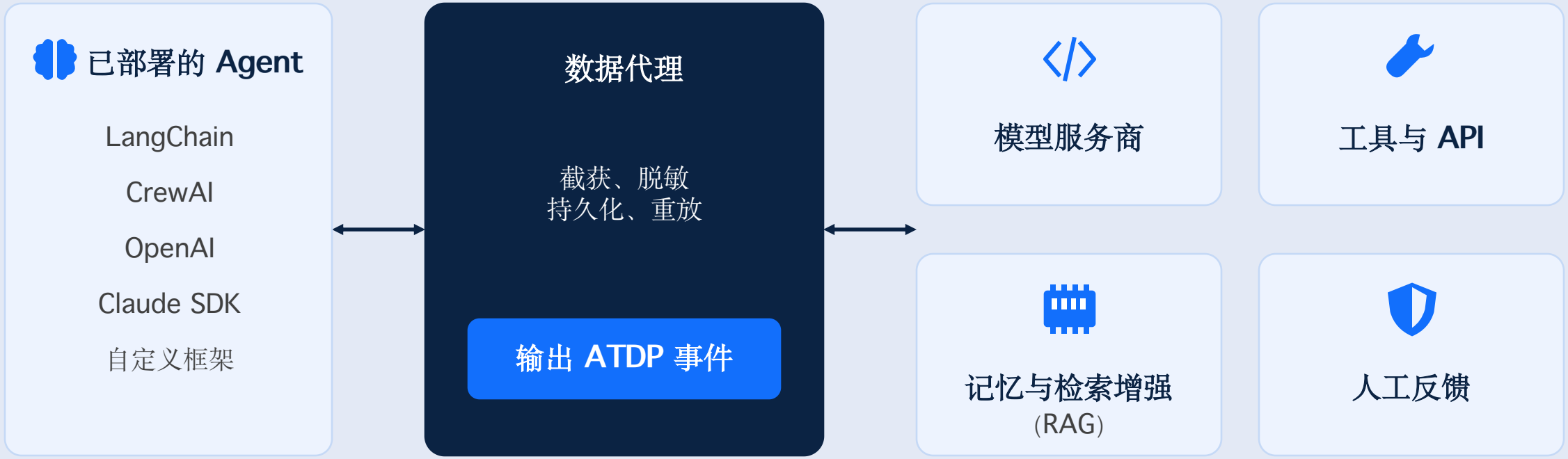
$$\tau = (e_1, e_2, \ldots, e_T)$$

$$e_t = \langle o_t,\ h_t,\ a_t,\ y_t,\ r_t,\ m_t \rangle$$

- o 观测：工具输出、检索结果、环境状态
- h 隐状态：计划、草稿、推理过程
- a 动作：工具调用、消息、记忆写入
- y 结果：工具返回、用户修改 / 重试
- r 奖励：标量、评语或延迟反馈
- m 元数据：租户、版本、成本、治理信息

全链路 Agent 数据代理

> 统一数据代理跨框架、跨服务商采集 Agent 交互，并支持脱敏、持久化与重放。



Agent 演化控制平面

> 基于轨迹统计，决定改什么、何时更新，以及如何评估或回滚每次改进。

持续演化的 Agent

$$\mathcal{A}_t = \langle \pi_{\theta_t}, \mathcal{H}_{\psi_t}, \mathcal{M}_t, \mathcal{T}_t, \mathcal{G}_t \rangle$$

$$u^\star = \arg\max_{u \in \mathcal{U}} [\mathcal{J}_{\mathcal{A}}(u \mid \mathcal{A}_t, \mathcal{D}_t)]$$

记忆更新规范

反复缺失的事实 · 具体流程中的经验。

工具接口规范变更

参数结构、权限与描述。

执行框架 / 技能演化

工具路由、检索结果格式、开发者消息结构。

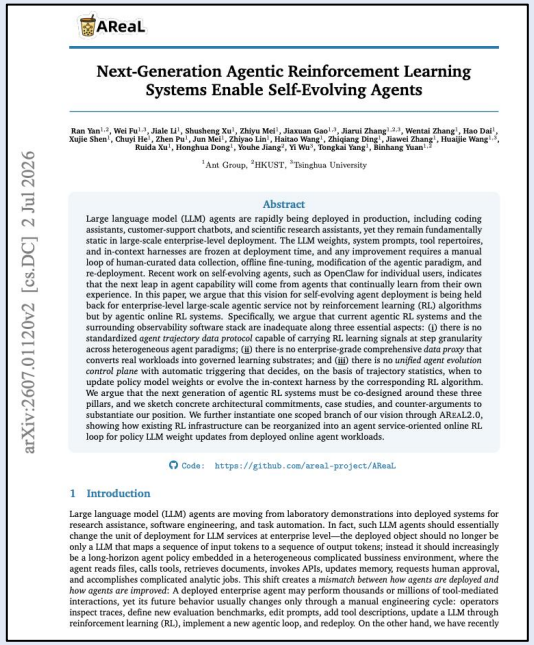
策略模型权重更新

SFT · DPO · 同策略强化学习 · 过程奖励学习 · 蒸馏。

回滚 / 保持不变

常规可选动作，而非应急措施。

AReaL 2.0：Agent 在线强化学习



AReaL

Next-Generation Agentic Reinforcement Learning Systems Enable Self-Evolving Agents

Ran Yan[1,2], Wei Fu[1,3], Jiale Li[1], Shusheng Xu[1], Zhiyu Mei[1], Jiaxuan Gao[1,3], Jiarui Zhang[1,2,3], Wentai Zhang[1], Hao Dai[1], Xujie Shen[1], Chuyi He[1], Zhen Pu[1], Jun Mei[1], Zhiyao Lin[1], Haitao Wang[1], Zhiqiang Ding[1], Jiawei Zhang[1], Huajie Wang[1,3], Ruida Xu[1], Hongbua Dong[1], Youhe Jiang[2], Yi Wu[3], Tongkai Yang[1], Binhang Yuan[1,2]

[1]Ant Group, [2]HKUST, [3]Tsinghua University

arXiv:2607.01120v2 [cs.DC] 2 Jul 2026

Abstract

Large language model (LLM) agents are rapidly being deployed in production, including coding assistants, customer-support chatbots, and scientific research assistants, yet they remain fundamentally static in large-scale enterprise-level deployment. The LLM weights, system prompts, tool repertoires, and in-context harnesses are frozen at deployment time, and any improvement requires a manual loop of human-curated data collection, offline fine-tuning, modification of the agentic paradigm, and re-deployment. Recent work on self-evolving agents, such as OpenClaw for individual users, indicates that the next leap in agent capability will come from agents that continually learn from their own experience. In this paper, we argue that this vision for self-evolving agent deployment is being held back for enterprise-level large-scale agentic service not by reinforcement learning (RL) algorithms but by agentic online RL systems. Specifically, we argue that current agentic RL systems and the surrounding observability software stack are inadequate along three essential aspects: (i) there is no standardized *agent trajectory data protocol* capable of carrying RL learning signals at step granularity across heterogeneous agent paradigms; (ii) there is no enterprise-grade comprehensive *data proxy* that converts real workloads into governed learning substrates; and (iii) there is no *unified agent evolution control plane* with automatic triggering that decides, on the basis of trajectory statistics, when to update policy model weights or evolve the in-context harness by the corresponding RL algorithm. We argue that the next generation of agentic RL systems must be co-designed around these three pillars, and we sketch concrete architectural commitments, case studies, and counter-arguments to substantiate our position. We further instantiate one scoped branch of our vision through AReaL2.0, showing how existing RL infrastructure can be reorganized into an agent service-oriented online RL loop for policy LLM weight updates from deployed online agent workloads.

Code: https://github.com/areal-project/AReaL

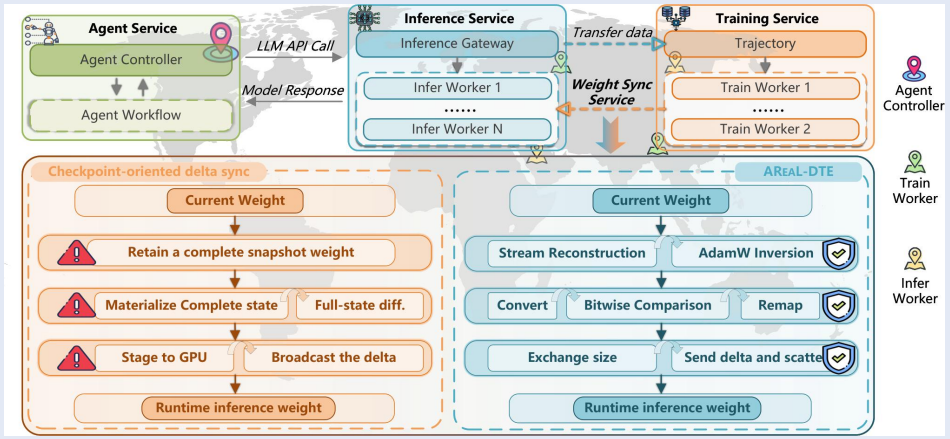
1 Introduction

Large language model (LLM) agents are moving from laboratory demonstrations into deployed systems for research assistance, software engineering, and task automation. In fact, such LLM agents should essentially change the unit of deployment for LLM services at enterprise level—the deployed object should no longer be only a LLM that maps a sequence of input tokens to a sequence of output tokens; instead it should increasingly be a long-horizon agent policy embedded in a heterogeneous complicated bussiness environment, where the agent reads files, calls tools, retrieves documents, invokes APIs, updates memory, requests human approval, and accomplishes complicated analytic jobs. This shift creates a *mismatch between how agents are deployed and how agents are improved:* A deployed enterprise agent may perform thousands or millions of tool-mediated interactions, yet its future behavior usually changes only through a manual engineering cycle: operators inspect traces, define new evaluation benchmarks, edit prompts, add tool descriptions, update a LLM through reinforcement learning (RL), implement a new agentic loop, and redeploy. On the other hand, we have recently

[arXiv 2607.01120]

AReaL 系统优化：AReaL-DTE

面向 Agent 在线强化学习的稀疏策略权重传输



策略同步耗时

11.58–12.23 秒

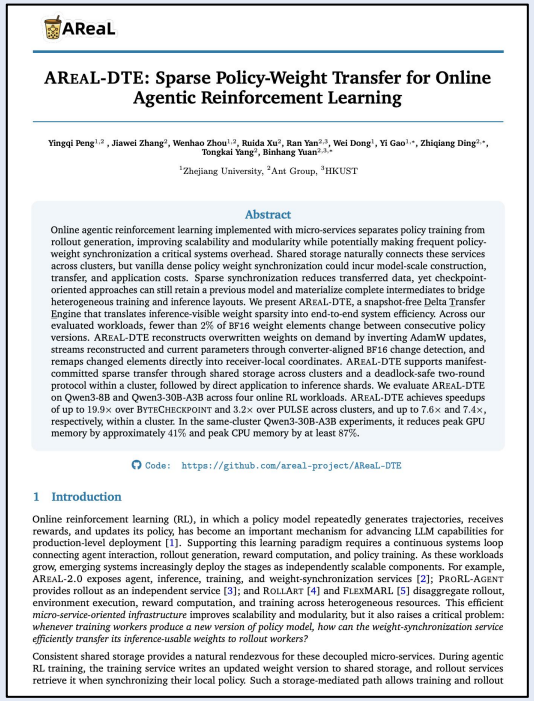
ByteCheckpoint-NCCL：82.67–89.48 秒
Qwen3-30B-A3B，同集群部署

策略权重频繁刷新

在已评估负载中，发生变化的 BF16 权重不足 2%，而全量刷新仍需处理整个模型。

无需快照的增量传输引擎

通过 AdamW 逆运算重建旧权重；融合格式转换与 BF16 变化检测，将稀疏更新重映射至推理分片。



AReaL

AReaL-DTE: Sparse Policy-Weight Transfer for Online Agentic Reinforcement Learning

Yingqi Peng[1,2], Jiawei Zhang[2], Wenhao Zhou[1,2], Ruida Xu[2], Ran Yan[2,3], Wei Dong[1], Yi Gao[1,*], Zhiqiang Ding[2,*], Tongkai Yang[2], Binhang Yuan[2,3,*]

[1]Zhejiang University, [2]Ant Group, [3]HKUST

Abstract

Online agentic reinforcement learning implemented with micro-services separates policy training from rollout generation, improving scalability and modularity while potentially making frequent policy-weight synchronization a critical systems overhead. Shared storage naturally connects these services across clusters, but vanilla dense policy weight synchronization could incur model-scale construction, transfer, and application costs. Sparse synchronization reduces transferred data, yet checkpoint-oriented approaches can still retain a previous model and materialize complete intermediates to bridge heterogeneous training and inference layouts. We present AReaL-DTE, a snapshot-free Delta Transfer Engine that translates inference-visible weight sparsity into end-to-end system efficiency. Across our evaluated workloads, fewer than 2% of BF16 weight elements change between consecutive policy versions. AReaL-DTE reconstructs overwritten weights on demand by inverting AdamW updates, streams reconstructed and current parameters through converter-aligned BF16 change detection, and remaps changed elements directly into receiver-local coordinates. AReaL-DTE supports manifest-committed sparse transfer through shared storage across clusters and a deadlock-safe two-round protocol within a cluster, followed by direct application to inference shards. We evaluate AReaL-DTE on Qwen3-8B and Qwen3-30B-A3B across four online RL workloads. AReaL-DTE achieves speedups of up to 19.9× over ByteCheckpoint and 3.2× over PULSE across clusters, and up to 7.6× and 7.4×, respectively, within a cluster. In the same-cluster Qwen3-30B-A3B experiments, it reduces peak GPU memory by approximately 41% and peak CPU memory by at least 87%.

Code: https://github.com/areal-project/AReaL-DTE

1 Introduction

Online reinforcement learning (RL), in which a policy model repeatedly generates trajectories, receives rewards, and updates its policy, has become an important mechanism for advancing LLM capabilities for production-level deployment [1]. Supporting this learning paradigm requires a continuous systems loop connecting agent interaction, rollout generation, reward computation, and policy training. As these workloads grow, emerging systems increasingly deploy the stages as independently scalable components. For example, AReaL-2.0 exposes agent, inference, training, and weight-synchronization services [2]; PRORL-AGENT provides rollout as an independent service [3]; and ROLLART [4] and FLEXMARL [5] disaggregate rollout, environment execution, reward computation, and training across heterogeneous resources. This efficient *micro-service-oriented infrastructure* improves scalability and modularity, but it also raises a critical problem: *whenever training workers produce a new version of policy model, how can the weight-synchronization service efficiently transfer its inference-usable weights to rollout workers?*

Consistent shared storage provides a natural rendezvous for these decoupled micro-services. During agentic RL training, the training service writes an updated weight version to shared storage, and rollout services retrieve it when synchronizing their local policy. Such a storage-mediated path allows training and rollout

[arXiv 2608.00455]

GPU 峰值显存降低约 41%

CPU 内存降低至少 87%

AReaL-AutoPilot：全生命周期管理

目标：Agent 控制层根据强化学习负载描述生成经验证的部署方案，启动并监控分布式系统，持续调整系统策略。

1 负载描述
模型、算法、数据集、资源约束与目标

→

2 方案验证
启动前验证并行、布局、批处理与采样 / 训练配比

→

3 配置与启动
在集群上部署分布式强化学习系统

→

4 运行监控
吞吐量、陈旧度、内存、慢节点与收敛

→

5 自适应调整
随模型、负载及硬件变化，调整系统策略

持续自适应——方案随运行演进，始终面向当前状态

借鉴 ARGUS · 先验证，再执行

- 分配 GPU 前，先验证部署方案：内存可行性、并行合法性与通信拓扑。
- 被拒绝的方案返回反例，让 Agent 明确违反的约束，而非事后分析崩溃日志。

借鉴 Autopoiesis · 边运行，边适应

- 强化学习后训练具有非平稳性：策略改进带来采样长度及最优配置的变化。
- 控制平面持续观测执行并在运行中改写系统策略，将双平面分离用于训练。

05

AReaL-RSI
递归自我改进

数据、执行框架与模型的联合迭代

AReaL RSI：学习闭环

面向数学解题，联合迭代数据、执行框架与模型

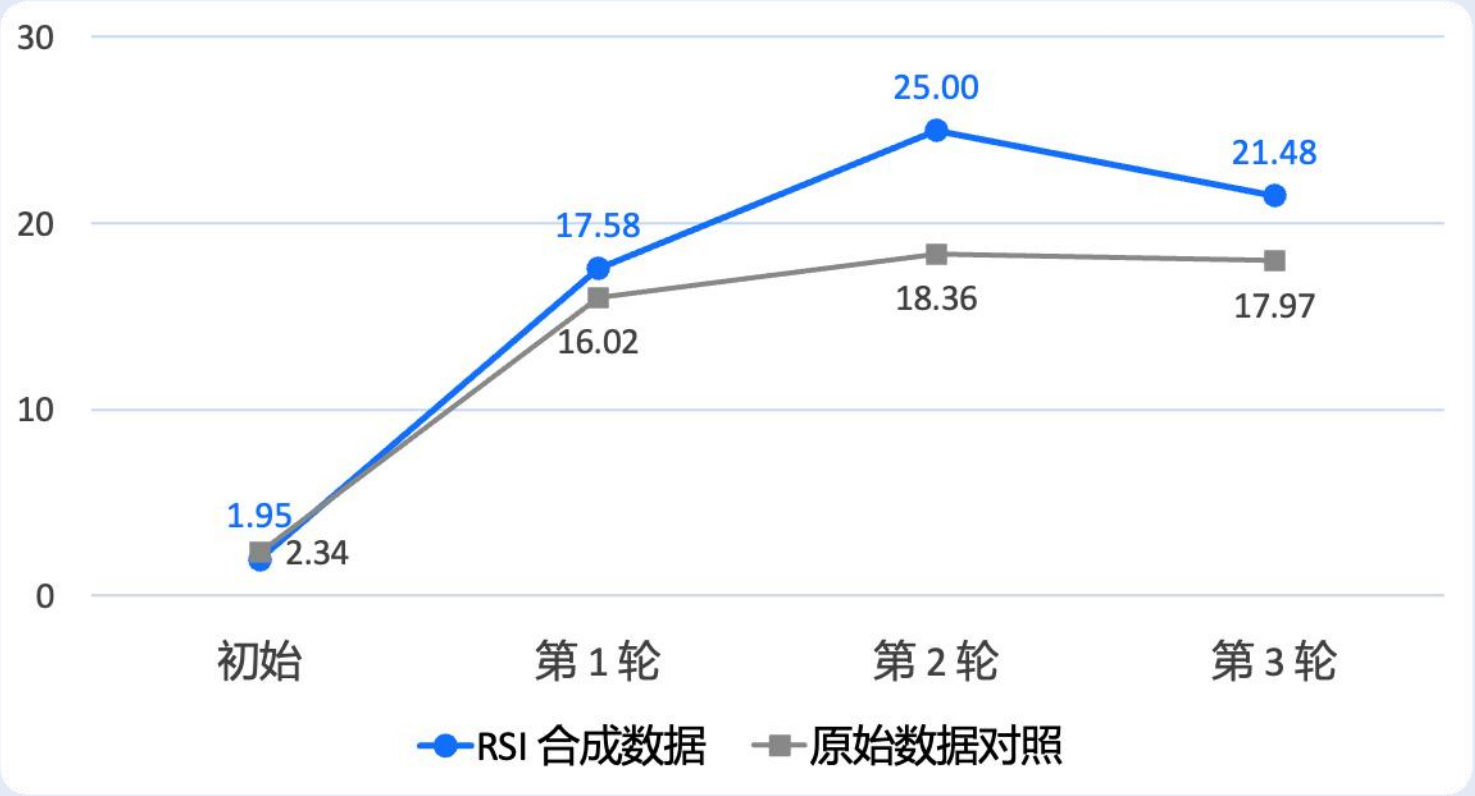
基础模型：Qwen2.5-Math-1.5B；执行框架：Claude Code



每轮：512 道训练题，2 个训练轮次 / 32 步；固定测试集包含 256 道题

AReaL RSI：初步结果

合成数据与强化学习提升数学解题准确率



25.00%

第 2 轮提升 6.64 个百分点

相较于同轮对照组

执行框架搜索

所有候选方案均未达到采纳门槛。

保留最优版本

第 3 轮效果下降后，保留第 2 轮版本。

基于固定 256 题测试集的初步实验；独立的初始评测存在波动。

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

总结

AReaL 连接可扩展的强化学习基础设施与 Agent 在线学习，以评测和版本控制支撑持续改进。

可扩展强化学习

开放、模块化的基础设施，支持异步强化学习、长程 Agent 及多种硬件上的高效训练。

在线学习

轨迹采集、数据代理与演化控制平面连接 Agent 交互、评测、训练和版本化部署。

自我改进

初步数学实验表明，合成数据与强化学习带来收益。评测和版本管理帮助迭代过程保留最优结果。

AReaL 代码仓库
github.com/areal-project/AReaL

个人主页
binhangyuan.github.io/site

袁彬航 香港科技大学 AReaL
小红书号：95876982936

谢谢!